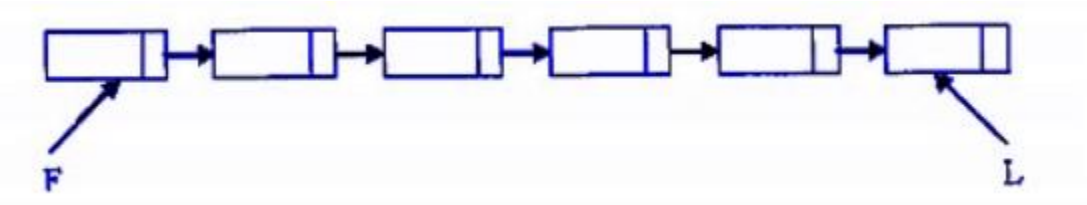
Linked List Tutorial

Q.1

Consider a singly linked list of the form where F is a pointer to the first element in the linked list and L is the pointer to the last element in the list.



The time of which of the following operations depend on the length of the list?

A) Delete the last element of the list
B) Delete the first element of the list
C) Add an element after the last element of the list.
D) Interchange the first two elelkments of the list.

Solution: A)

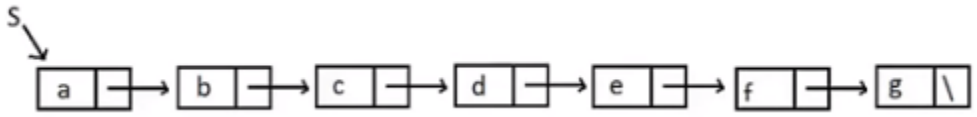
option(A): Delete the last element of the list depends on the length of the list because you need to find the
last before element and last before element not accessible from the L pointer because it single list not double list
For finding last before element we need to linear search the linked list and so it depends on the length of the linked list.
After finding you need to perform the below operation
(Last before Node)->next = NULL

option(B) : Deleting first Element does not depend on the length of the list you can simple delete by using the F pointer
i.e, F = F -> next;

option(C) : Add an element at the end of the list
we know the Address of the last Element of the list i.e L . So Just perform the following operations
L->next=new node
now L=new node

option(D) : Interchange the first two elements of the list
this can be performed using a temp node and does not depends on the length of the list.
Temp =F
F=F->next
Temp->next=F->next
F->next=Temp

Q.2) Consider the below given linked list:



Perform the following steps and identify the final output after performing all the steps:

1. Struct node *P;
2. P = S → next → next → next;
3. P → next → next → next → next = S → next → next;
4. S → next → next = P → next → next → next;
5. Print(P → next → next → next → next → next → next → next→ data)

https://www.youtube.com/watch?v=lfFfw4MeT7Y

Q.3) What is the output of following function for start pointing to first node of following linked list? 1->2->3->4->5->6

```
void fun(struct node* start)
{
  if(start == NULL)
    return;
  printf("%d  ", start->data);

  if(start->next != NULL )
    fun(start->next->next);
  printf("%d  ", start->data);
}
```

Q.4) The following c function takes a singly linked list as input argument . It modified the list by moving the last element to the front of the list and returns the modified list. Some part of the code is left blank. Identify the appropriate code from the following options.

typedef struct node { int value; struct node *next; } Node;

Node *move_to-front(Node *head)

{

  Node *p, *q;

  if ((head == NULL) || (head -> next == NULL)) return head;

  q = NULL; p = head;

  while (p->next != NULL)

  {

    q=p;

    p=p->next;

  }

  _____

  return head;

}

a) q = NULL; p->next = head; head = p;

b) q->next = NULL; head = p; p->next = head;

c) head = p; p->next=q; q->next = NULL;

d) q->next = NULL; p ->next = head; head = p;

https://www.youtube.com/watch?v=WZohENYqY4A

Q.5) The following C function takes a singly linked list of integers as a parameter and re-arrange the elements of the list. The list is represented as pointer to a structure. The function is called with the list containing the integers 1,2,3,4,5,6,7 in the given order. What will be the contents of the list after the function completes execution?

```
struct node {
int value;
struct node * next;
};
Void rearrange (struct node * list) {
struct node * p, * q;
int temp;
if (!list || !list  -> next) return;
p = list; q = list  -> next;
while (q) {
temp = p  -> value; p -> value = q -> value;  q -> value = temp;
p = q -> next
q = p? p -> next : 0;
}
}
```

A) 1,2,3,4,5,6,7

B) 2,1,4,3,6,5,7

C) 1,3,2,5,4,7,6

D) 2,3,4,5,6,7,1

Q.6) Consider the c code fragment given below.

```
Typedef struct node{
        int data
        node *next;
}node;

Void join(node* m, node* n )
{
node* p = n;
while(p->next !=NULL)
{
    p = p->next;
}
 p->next = m;
}
```

Assuming that m and n point to valid NULL-terminated linked lists, invocation of op function will

a) Always appends list ‘m’ at the end of list ‘n’ for all inputs.
b) Either cause a null pointer dereference or append list m to the end of the list n.
c) Cause a null pointer dereference for all inputs.
d) Append list ‘n’at the end of list ‘m’ for all inputs

Solution: b)

Coding Based Questions

Q.1:

Delete Middle of Linked List

Given a singly linked list, delete **middle** of the linked list. For example, if given linked list is 1->2->**3**->4->5 then linked list should be modified to 1->2->4->5.
If there are **even** nodes, then there would be **two middle** nodes, we need to delete the second middle element. For example, if given linked list is 1->2->3->4->5->6 then it should be modified to 1->2->3->5->6.
If the input linked list is NULL or has 1 node, then it should return NULL

Example 1:

Input:

LinkedList: 1->2->3->4->5

Output: 1 2 4 5

Example 2:

Input:

LinkedList: 2->4->6->7->5->1

Output: 2 4 6 5 1

Solution: https://www.youtube.com/watch?v=3HRCz3w-7rw

Que.2 Detect Loop in linked list

Accolite Adobe Amazon D-E-Shaw Hike Lybrate
Mahindra Comviva MakeMyTrip MAQ Software OYO Rooms
Paytm Qualcomm Samsung SAP Labs Snapdeal Veritas
VMWare Walmart

Example 1:

Input:
N = 3
value[] = {1,3,4}
X = 2
Output: 1
Explanation: The link list looks like
1 -> 3 -> 4
^ |
|____|
A loop is present. If you remove it
successfully, the answer will be 1.

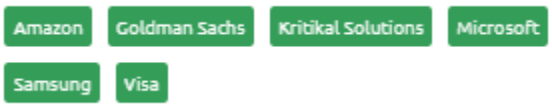
Example 2:

Input:
N = 4
value[] = {1,8,3,4}
X = 0
Output: 1
Explanation: The Linked list does not
contains any loop.

Solution: https://www.youtube.com/watch?v=zbozWoMgKW0z

Q.3 Delete without head pointer

You are given a pointer/ reference to the node which is to be deleted from the linked list of **N** nodes. The task is to delete the node. Pointer/ reference to head node is not given.
Note: No head reference is given to you. It is guaranteed that the node to be deleted is not a tail node in the linked list.



Example 1:

Input:

N = 2

value[] = {1,2}

node = 1

Output: 2

Explanation: After deleting 1 from the

linked list, we have remaining nodes

as 2.

Example 2:

Input:

N = 4

value[] = {10,20,4,30}

node = 20

Output: 10 4 30

Explanation: After deleting 20 from
the linked list, we have remaining
nodes as 10, 4 and 30.

Solution: https://www.youtube.com/watch?v=PApPhb61QcQ

Q.4 **Intersection Point in Y Shaped Linked Lists**

Example 1:

Input:

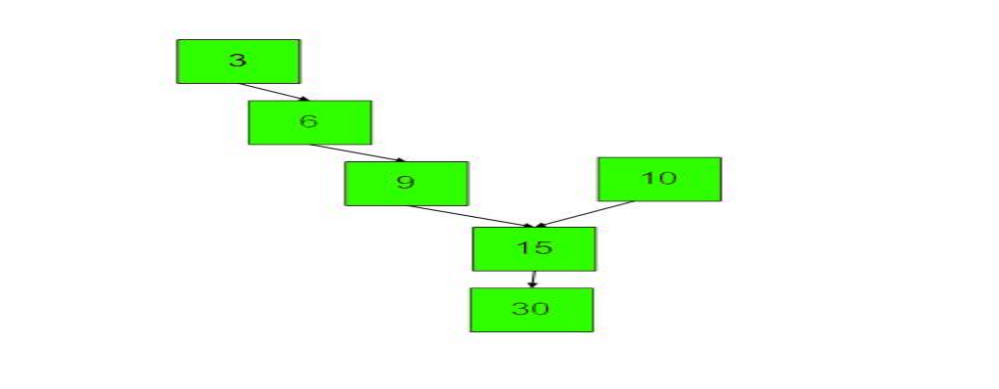
LinkList1 = 3->6->9->common

LinkList2 = 10->common

common = 15->30->NULL

Output: 15

Explanation:

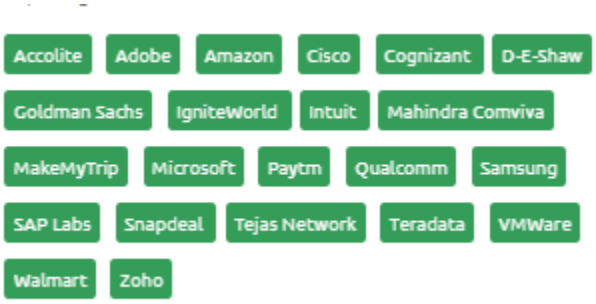


Solution: https://www.youtube.com/watch?v=_7byKXAhxyM

https://www.youtube.com/watch?v=lunWlOjj0Cg

Q.5 Reverse a linked list

Given a pointer to the head node of a linked list, the task is to reverse the linked list *(without using any auxiliary space)*.



If auxiliary space is allowed, one can use a Stack to generate the reversed list. We can discuss this point, if the stack data-structure has been covered in the class.

Examples:

Input1: 1->2->3->4->NULL **Output1**: 4->3->2->1->NULL

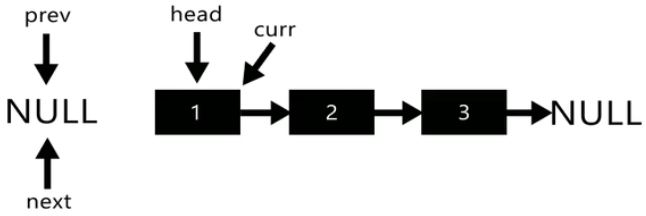
Input2: NULL **Output2**: NULL

Input3: 1->NULL **Output3**: 1->NULL

Iterative Method

1. Initialize three pointers prev as NULL, curr as head and next as NULL.
2. Iterate through the linked list. In loop, do the following.
   // Before changing next of current,
   // store next node
   next = curr->next
   // Now change next of current
   // This is where actual reversing happens
   curr->next = prev

// Move prev and curr one step forward
prev = curr
curr = next




```
while (current != NULL)
    {
        next  = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }
    *head_ref = prev;
```

Q.6. Given a singly linked list of characters, write a function that returns true if the given list is a palindrome, else false. *<Auxiliary space is not allowed>*

st

Accolite Adobe Amazon KLA Tencor Kritikal Solutions MakeMyTrip Microsoft Snapdeal Yodlee Infotech

If auxiliary space is allowed, we can use a Stack to store the elements of the linked list. Then, we can compare the elements in the Stack with the elements of the original linked list. If all the elements match, it is a palindrome, else not. We can discuss this point, if the stack data-structure has been covered in the class.

Iterative solution: By reversing the list

This method takes O(n) time and O(1) extra space.

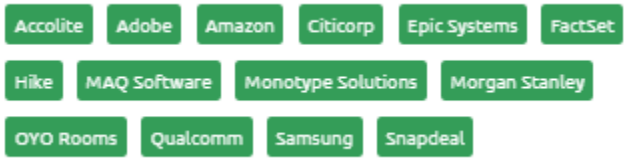
1) Get to the middle of the linked list.

2) Reverse the second half of the linked list. ← *Refer to the previous question*

3) Check if the first half and second half are identical.

4) If there is a need to print the original list, construct the original linked list by reversing the second half again and attaching it back to the first half.

Q.7 How do you find the third node (or the k^{th} node) from the end in a singly linked list?



The main challenge here is to solve the problem in just one pass. That means, you can not traverse the linked list again and you cannot traverse backward because it's a singly linked list.

Here, we'll use the **tortoise and hare algorithm** to solve this problem. This is the same algorithm that we have used to find the middle element of the linked list in one pass.

The algorithm uses two pointers, fast and slow. The slow pointer starts when the fast pointer is reached to the k^{th} node. For example, in order to find the 3rd element from the last, the second pointer should start when the first pointer has reached the third element.

After that, both pointers will keep moving one step at a time until the first pointer points to null, which signals the end of the linked list. At this time, the second pointer is pointing to the 3rd or k^{th} node from the last.

Que.4 Check If Circular Linked List

Given **head**, the head of a singly linked list, find if the linked list is circular or not.

Microsoft, Amazon

Example 1:

Input:

LinkedList: 1->2->3->4->5

(the first and last node is connected,

i.e. 5 --> 1)

Output: 1

Example 2:

Input:

LinkedList: 2->4->6->7->5->1

Output: 0

Your Task:
The task is to complete the function isCircular() which checks if the given linked list is circular or not. It should return true or false accordingly. (the driver code prints 1 if the returned values is true, otherwise 0)

Solution: **https://www.youtube.com/watch?v=dII_jB9S9Ew**

https://www.youtube.com/watch?v=ce2rnhkNLzU

Que.8 Remove duplicates from a sorted linked list

For example if the linked list is 11->11->11->21->43->43->60 then removeDuplicates() should convert the list to 11->21->43->60.

Algorithm: Traverse the list from the head (or start) node. While traversing, compare each node with its next node. If the data of the next node is the same as the current node then delete the next node. Before we delete a node, we need to store the next pointer of the node

```
/* The function removes duplicates from a sorted list */
void removeDuplicates(Node* head)
{
   /* Pointer to traverse the linked list */
```

```
    Node* current = head;

    /* Pointer to store the next pointer of a node to be deleted*/
    Node* next_next;

    /* do nothing if the list is empty */
    if (current == NULL)
    return;

    /* Traverse the list till last node */
    while (current->next != NULL)
    {
    /* Compare current node with next node */
    if (current->data == current->next->data)
    {
       /* The sequence of steps is important*/
       next_next = current->next->next;
       free(current->next);
       current->next = next_next;
    }
    else /* Only advance if no deletion */
    {
       current = current->next;
    }
    }
}
```

METHOD 2

Alternate Approach: Create a pointer ptr that will point towards the first occurrence of every element and another pointer temp which will iterate to every element and when the value of the previous pointer is not equal to the temp pointer, we will set ptr->next = temp. But remember, before setting ptr->next = temp, you should free all the subsequent nodes having the same value. *Refer to the first question for the deletion (freeing) step.*

With this approach, if any value is repeated multiple times (more than twice), all the subsequent nodes having the same value can be excluded together. i.e. unlike the first approach, we need to reassign the pointers here exactly once (even if a value is repeated more than twice).

Que.9 Remove duplicates from an unsorted linked list

For example if the linked list is 12->11->12->21->41->43->21 then remove Duplicates() should convert the list to 12->11->21->41->43.

METHOD 1 (Using two loops)

This is the simple way where two loops are used. Outer loop is used to pick the elements one by one and the inner loop compares the picked element with the rest of the elements.

```
/* Function to remove duplicates from a unsorted linked list */
void removeDuplicates(struct Node* start)
{
  struct Node *ptr1, *ptr2, *dup;
  ptr1 = start;

  /* Pick elements one by one */
  while (ptr1 != NULL && ptr1->next != NULL) {
    ptr2 = ptr1;
    /* Compare the picked element with rest of the elements */
    while (ptr2->next != NULL) {
      /* If duplicate then delete it */
```

```
            if (ptr1->data == ptr2->next->data) {
                /* sequence of steps is important here */
                dup = ptr2->next;
                ptr2->next = ptr2->next->next;
                delete (dup);
            }
            else /* This is tricky */
                ptr2 = ptr2->next;
        }
        ptr1 = ptr1->next;
    }
}
```

Time Complexity: $O(n^2)$

METHOD 2 (Using Sorting: Merge Sort on Linked List)

An efficient solution. *If merge sort has been covered in the lectures, we can discuss this solution else, we can skip.*

In general, Merge Sort is the best-suited sorting algorithm for sorting linked lists efficiently.

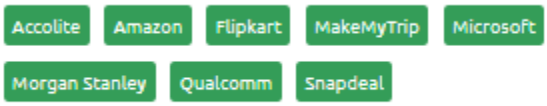
1) Sort the elements using Merge Sort. O(nLogn)

2) Remove duplicates in linear time using the algorithm for removing duplicates in sorted Linked List. O(n)

Please note that this method doesn't preserve the original order of elements.

Overall Time Complexity: O(nLogn)

Que.10 Add two numbers represented by linked lists



The sum list is a linked list representation of the addition of two input numbers **from the last.**
Example 1:

Input:

N = 2

valueN[] = {4,5}

M = 3

valueM[] = {3,4,5}

Output: 3 9 0

Explanation: For the given two linked list (4 5) and (3 4 5), after adding the two linked list resultant linked list will be (3 9 0).

Example 2:

Input:

N = 2

valueN[] = {6,3}

M = 1

valueM[] = {7}

Output: 7 0

Explanation: For the given two linked list (6 3) and (7), after adding the two linked list resultant linked list

will be (7 0).

Solution: https://www.youtube.com/watch?v=HzWKOp0C_aU

Que.10 Partitioning a linked list around a given value and keeping the original order

Given a linked list and a value x, partition it such that all nodes less than x come first, then all nodes with a value equal to x, and finally nodes with a value greater than or equal to x. Also, the original relative order of the nodes in each of the three partitions should be preserved.

Examples:

Input : 1->4->3->2->5->2->3, x = 3

Output: 1->2->2->3->3->4->5

Input : 1->4->2->10, x = 3

Output: 1->2->4->10

Input : 10->4->20->10->3, x = 3

Output: 3->10->4->20->10

Algorithm:

- Initialize first and last nodes of below three linked lists as NULL.
    1. Linked list of values smaller than x.
    2. Linked list of values equal to x.
    3. Linked list of values greater than x.
- Now iterate through the original linked list. If a node's value is less than x then append it at the end of the smaller list. If the value is equal to x, then at the end of the equal list. And if a value is greater, then at the end of the greater list.
- Now concatenate three lists.

Solution: https://www.youtube.com/watch?v=iLhRF71D3_w